

Recursividade, Merge Sort e Quick Sort

Guia de Estudos Comentado — Gabarito explicado passo a passo



Sobre este material

Este guia foi criado a partir da prova "**DSA - Avaliação: Recursividade, Merge Sort e Quick Sort**" (Turma 4) e das respostas discutidas em chat. Ele resolve **todas as 9 questões** em detalhe, com:

- explicação conceitual antes de cada resposta;
- execução do código **linha a linha**;
- diagramas ilustrativos das árvores de recursão, divisão e partição;
- tabelas de rastreamento (trace) para o algoritmo `merge()` ;
- justificativas completas para cada questão discursiva.

Como usar: leia a explicação de conceito, acompanhe o código, compare com o diagrama e só depois confira a resposta. Isso fixa o raciocínio, não apenas o resultado.

Material de apoio ao estudo · Baseado na prova enviada e no chat de resolução

Conceitos-base antes de começar

1. O que é recursão?

Uma função é **recursiva** quando ela chama a si mesma para resolver uma versão menor do mesmo problema. Toda função recursiva bem formada precisa de duas partes:

Caso-base: a condição que faz a recursão *parar* (sem ela, a função chamaria a si mesma para sempre e o programa quebraria).

Passo recursivo: a chamada da função a ela mesma, normalmente com uma entrada "menor" ou "mais simples" que a original, aproximando-a do caso-base.

Pensando em pilha de chamadas: cada chamada recursiva "empilha" uma nova execução. Quando o caso-base é atingido, a pilha começa a ser desempilhada — esse processo de desempilhar é chamado de **retorno** da recursão. Tudo que acontece antes da chamada recursiva ocorre durante a **descida**; tudo que acontece depois dela (inclusive combinar resultados) ocorre durante o **retorno**.

2. Slicing em Python (revisão rápida)

A forma geral do fatiamento é `vetor[inicio:fim:passo]`. O índice de início é incluído e o de fim não é incluído. Essa é a base de como o Merge Sort divide as listas: `lista[:meio]` pega a metade da esquerda e `lista[meio:]` pega a metade da direita — sem precisar saber os valores, apenas as posições.

3. Duas estratégias de "dividir para conquistar"

Merge Sort	Quick Sort
Divide a lista sempre ao meio (pela posição), não importa os valores.	Divide a lista de acordo com um pivô : menores, iguais e maiores que ele.
Todo o trabalho pesado (comparações) acontece na função <code>merge()</code> , durante o retorno da recursão.	Todo o trabalho pesado (comparações) acontece na função <code>particionar</code> , durante a descida da recursão.
A árvore de chamadas é sempre balanceada (profundidade $\sim \log n$).	A árvore de chamadas pode ficar desbalanceada se o pivô for mal escolhido (pior caso $\sim n$).

Com esses conceitos em mente, vamos resolver a prova questão por questão.

Questão 1 — O caminho até o caso-base

Parte 1: função `reduzir(n)`

```
n = 28

def reduzir(n):
    print(n)
    if n <= 1:
        return
    reduzir(n // 2)

reduzir(n)
```

Repare na ordem das instruções dentro da função: primeiro o `print(n)`, depois o teste do caso-base e só então a chamada recursiva. Isso significa que **o valor é impresso antes de a lista "descer" para o próximo nível**. Vamos simular chamada por chamada:

Chamada	n	print(n)	n <= 1 ?	Próxima chamada
1ª	28	28	não	<code>reduzir(28 // 2) = reduzir(14)</code>
2ª	14	14	não	<code>reduzir(14 // 2) = reduzir(7)</code>
3ª	7	7	não	<code>reduzir(7 // 2) = reduzir(3)</code>
4ª	3	3	não	<code>reduzir(3 // 2) = reduzir(1)</code>
5ª	1	1	sim	<code>return</code> (fim da recursão)

`print(n)` acontece
ANTES da chamada seguinte

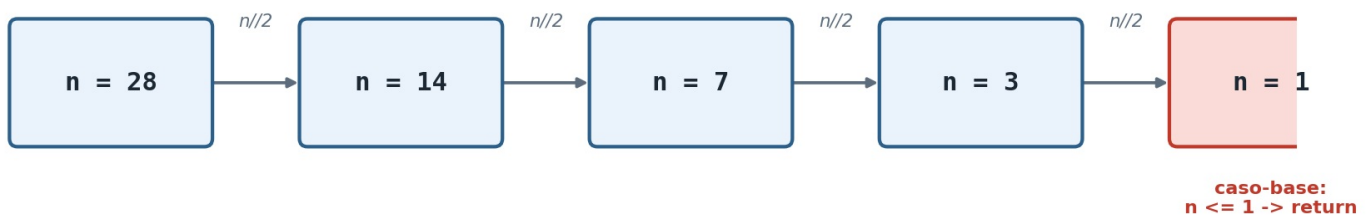


Figura 1 – Cadeia de chamadas de `reduzir(28)`: uma única chamada por nível, sempre dividindo n por 2.

Q1.A — Sequência impressa: `28 . 14 . 7 . 3 . 1`

Q1.B — Número de chamadas realizadas (incluindo o caso-base): **5**

Por que isso importa: cada chamada divide n por 2, então o número de chamadas até chegar a 1 é aproximadamente $\log_2(n)$. Para $n=28$, $\log_2(28) \approx 4,8$, arredondando para as 5 chamadas que contamos. Esse crescimento logarítmico é a mesma ideia por trás da eficiência do Merge Sort.

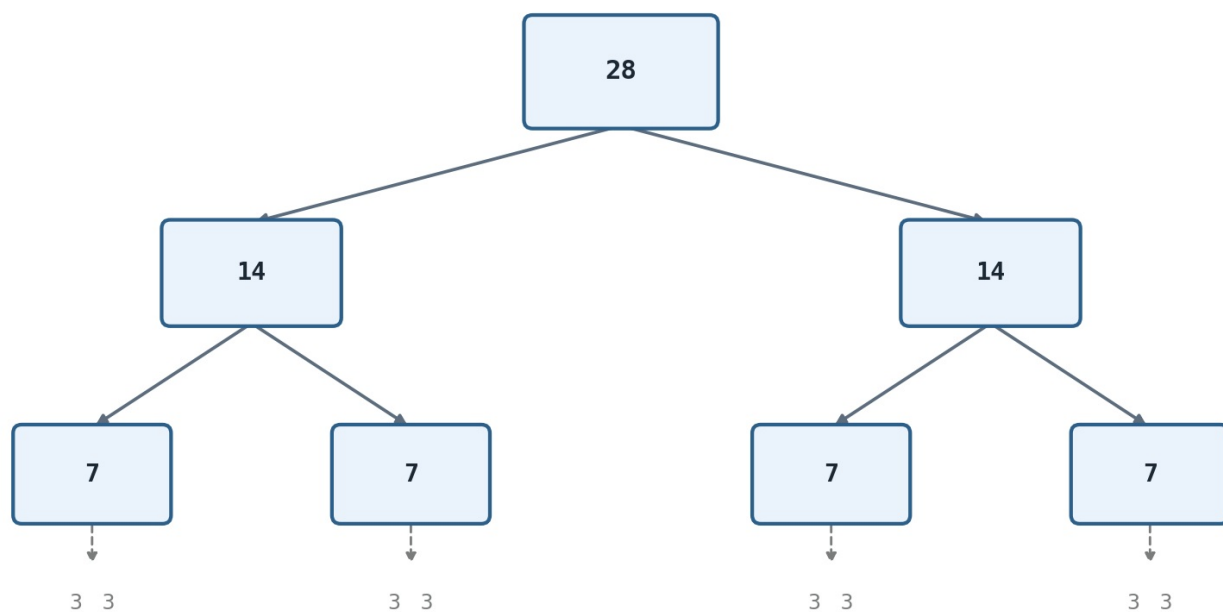
Parte 2: função `reduzir2(n)`

```
n = 28

def reduzir2(n):
    if n <= 1:
        return
    reduzir2(n // 2)
    reduzir2(n // 2)

reduzir2(n)
```

Aqui a diferença crucial é que a função chama a si mesma **duas vezes** a cada nível (as duas chamadas usam o mesmo valor `n // 2`, mas são duas execuções independentes da função, cada uma criando sua própria "sub-árvore" de chamadas). Isso significa que o número de chamadas não cresce de forma linear (uma por nível) e sim **dobra a cada nível**:



... e assim por diante, sempre DUAS chamadas por nível,
ate chegar ao caso-base ($n \leq 1$)

Figura 2 - reduzir2 forma uma árvore binária: cada nó gera dois filhos com o mesmo valor $n//2$.

Q1.C — A estrutura das chamadas de `reduzir2` é: **B) árvore binária, com duas chamadas menores em cada nível**

Por que isso importa: essa é exatamente a estrutura de chamadas do **Merge Sort** (duas chamadas recursivas por nível, uma para cada metade da lista) — e é por isso que o Merge Sort é analisado como uma árvore binária de divisão, como veremos na Questão 2.

Questão 2 — Dividindo o problema (Merge Sort)

```
V = [10, 4, 8, 2, 9, 3, 7, 5]
```

```
def merge_sort(lista):  
    if len(lista) <= 1:  
        return lista  
    meio = len(lista) // 2  
    esquerda = lista[:meio]  
    direita = lista[meio:]  
    esquerda = merge_sort(esquerda)  
    direita = merge_sort(direita)  
    return merge(esquerda, direita)
```

```
merge_sort(V)
```

A lista `V` tem 8 elementos, então `meio = 8 // 2 = 4`. Isso corta a lista **pela posição** (índice 4), não pelo valor dos elementos:

`esquerda = V[:4] = [10, 4, 8, 2]` | `direita = V[4:] = [9, 3, 7, 5]`

Um detalhe importante da ordem de execução: o código chama `merge_sort(esquerda)` **antes** de chamar `merge_sort(direita)`. Como Python executa uma chamada até o fim antes de seguir para a próxima linha, toda a divisão do lado esquerdo (até o caso-base) acontece primeiro, e só depois o lado direito começa a ser dividido. Por isso, no nível 2, a ordem das quatro listas segue a ordem: primeiro as duas metades de `esquerda`, depois as duas metades de `direita`.

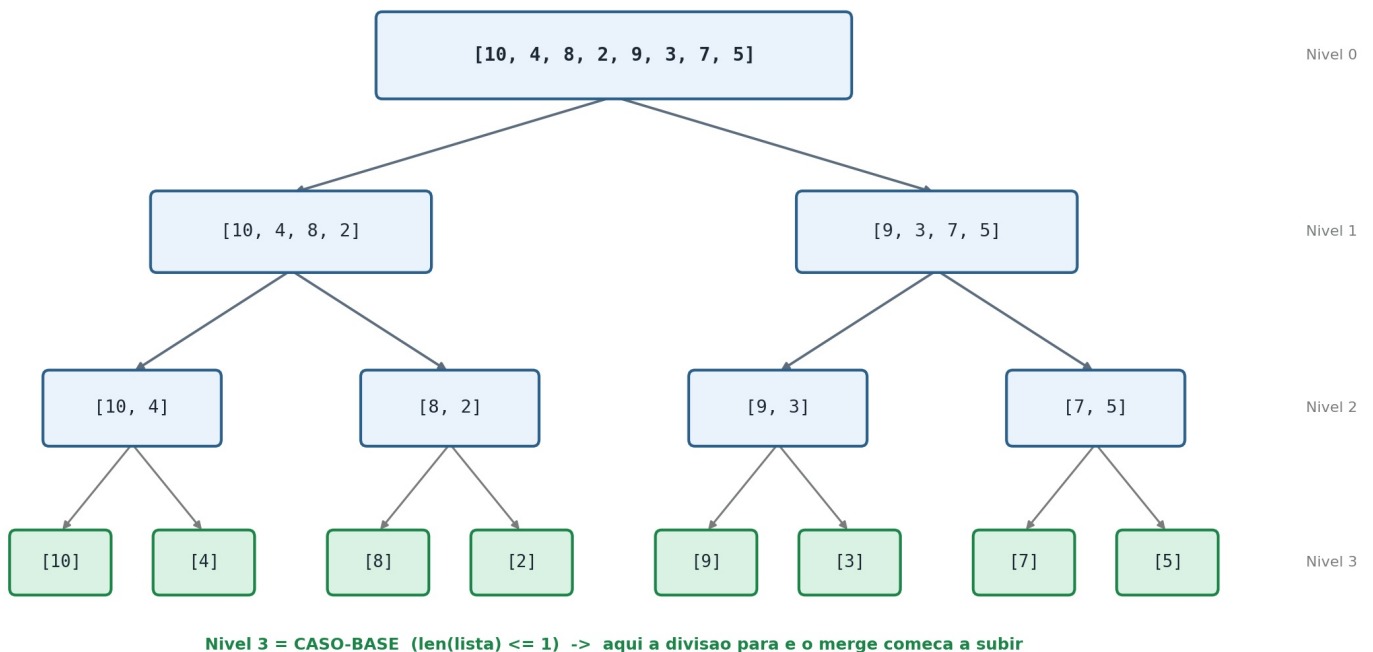


Figura 3 – Árvore de divisão completa. A prova pede apenas até o Nível 2 (destacado em azul); o Nível 3 (verde) é mostrado para completar o raciocínio.

Q2.A Nível 1 - esquerda: `[10, 4, 8, 2]`

Q2.B Nível 1 - direita: `[9, 3, 7, 5]`

Q2.C Nível 2 - 1ª lista: `[10, 4]`

Q2.D Nível 2 - 2ª lista: `[8, 2]`

Q2.E Nível 2 - 3ª lista: `[9, 3]`

Q2.F Nível 2 - 4ª lista: `[7, 5]`

Q2.G — Verdadeiro ou Falso

Afirmação	Resposta	Por quê
a) Os valores armazenados determinam o ponto em que a lista é dividida.	F	A divisão usa <code>len(lista)//2</code> , ou seja, a posição (índice), nunca o valor dos elementos.
b) As quatro listas do nível 2 possuem dois elementos.	V	<code>[10,4]</code> , <code>[8,2]</code> , <code>[9,3]</code> e <code>[7,5]</code> têm exatamente 2 elementos cada.
c) Nenhum elemento precisou ser ordenado para	V	A fase de divisão só fatia a lista; nenhuma comparação entre valores acontece

realizar essas divisões.		aqui — isso só ocorrerá dentro do <code>merge()</code> .
d) Nesse nível, o caso-base <code>len(lista) <= 1</code> já foi alcançado.	F	No nível 2 as listas ainda têm 2 elementos; o caso-base só é alcançado no nível 3, quando sobra 1 elemento por lista.

Questão 3 — Descida e retorno da recursão

```
V = [10, 4, 8, 2]
```

```
def merge(esquerda, direita):
    resultado = []
    i = j = 0
    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i]); i += 1
        else:
            resultado.append(direita[j]); j += 1
    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])
    return resultado
```

Vamos calcular os três merges pedidos, aplicando o algoritmo exatamente como está escrito:

Q3.A — merge([10], [4])

$i=0, j=0$: compara 10 (esquerda) com 4 (direita) $\rightarrow 10 \leq 4$ é falso $\rightarrow$ copia 4 (direita), $j=1$. Como j chegou ao fim de `direita`, o laço para. Sobrou [10] em esquerda, que é anexado no final com `extend`.

Resultado: [4, 10]

Q3.B — merge([8], [2])

Mesma lógica: $8 \leq 2$ é falso $\rightarrow$ copia 2 (direita); sobra [8], que é anexado ao final.

Resultado: [2, 8]

Q3.C — merge([4,10], [2,8])

Passo	i	j	esquerda[i]	direita[j]	comparação	copia	resultado parcial
1	0	0	4	2	$4 \leq 2$? não	2 (direita)	[2]
2	0	1	4	8	$4 \leq 8$? sim	4 (esquerda)	[2, 4]
3	1	1	10	8	$10 \leq 8$? não	8 (direita)	[2, 4, 8]

Nesse ponto $j=2$ chegou ao fim de `direita`; o laço encerra e o restante de `esquerda` ([10]) é anexado.

Resultado final: [2, 4, 8, 10]

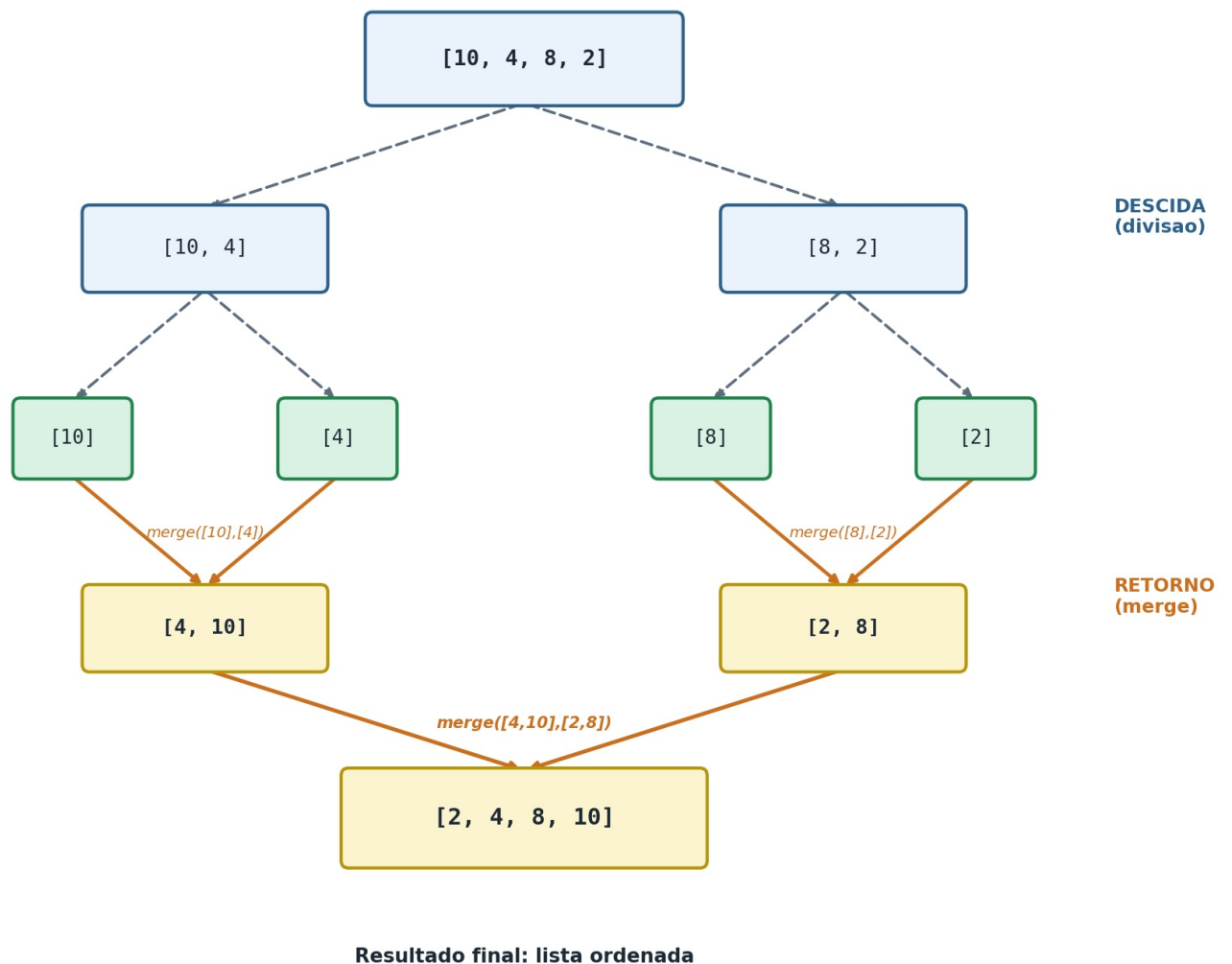


Figura 4 - O "diamante" da recursão: a divisão desce (setas tracejadas) até o caso-base, e o merge sobe (setas laranjas) reconstruindo a lista ordenada.

Associação: A = divisão; B = caso-base; C = merge

Q3.D Acontece durante o retorno da recursão: **C (merge)**

Q3.E É atingido quando a lista possui um único elemento: **B (caso-base)**

Q3.F Predomina durante a descida das chamadas: **A (divisão)**

Q3.JUSTIFICATIVA

Durante a **descida** da recursão, o código só executa a parte de divisão: cada chamada de `merge_sort` corta a lista ao meio e chama a si mesma novamente, sem combinar nada ainda — por isso a divisão (A) predomina nessa fase, até que o caso-base `len(lista) <= 1` (B) seja atingido em cada ramo. A partir daí, a recursão começa a **retornar**: cada chamada que estava "pausada" esperando o resultado das chamadas de `merge_sort(esquerda)` e `merge_sort(direita)` agora recebe essas listas já ordenadas e as combina com `merge()` (C). Ou seja, a descida constrói o caminho até os elementos isolados, e o retorno é exatamente onde o trabalho de ordenação (as comparações) de fato acontece, subindo nível a nível até reconstituir a lista inteira ordenada.

Questão 4 — Acompanhe merge()

```
esquerda = [1, 6]
direita = [3, 5]
print(merge(esquerda, direita))
```

Ambas as listas já estão ordenadas, exatamente como o `merge()` espera. Vamos seguir os ponteiros `i` (esquerda) e `j` (direita) passo a passo:

Passo	Comparação	Copia	i	j	resultado
1	esquerda[0]=1 × direita[0]=3 → 1≤3? sim	1	1	0	[1]
2	esquerda[1]=6 × direita[0]=3 → 6≤3? não	3	1	1	[1, 3]
3	esquerda[1]=6 × direita[1]=5 → 6≤5? não	5	1	2	[1, 3, 5]

Como `j=2` agora é igual a `len(direita)`, o laço `while` encerra. O elemento restante de `esquerda` a partir do índice 1 (ou seja, `[6]`) é anexado com `extend`.

Q4.A Passo 1: compare `1 × 3`; copie `1`; `i=1`; `j=0`; resultado= `[1]`

Q4.B Passo 2: compare `6 × 3`; copie `3`; `i=1`; `j=1`; resultado= `[1, 3]`

Q4.C Passo 3: compare `6 × 5`; copie `5`; `i=1`; `j=2`; resultado= `[1, 3, 5]`

Q4.D Qual lista terminou primeiro? `direita` (`[3, 5]`)

Q4.E Qual elemento ainda precisa ser acrescentado? `6`

Q4.F Resultado final: `[1, 3, 5, 6]`

Por que isso importa: repare que assim que uma das duas listas acaba, não é preciso continuar comparando — o restante da outra lista já está ordenado e pode ser "colado" direto no final com `extend()`. Essa é a parte que a Questão 5 vai testar propositalmente.

Questão 5 — Quando uma condição é quebrada

```
esquerda = [6, 1] # atenção: NÃO está ordenada!
direita = [3, 5]
print(merge(esquerda, direita))
```

Aqui está a pegadinha: a lista `esquerda` não está ordenada. O código de `merge()` não verifica isso — ele simplesmente confia que ambas as entradas já vêm ordenadas e compara sempre a **posição atual** de cada ponteiro (`esquerda[i]` e `direita[j]`), nunca a lista inteira.

Passo	Comparação	Copia	i	j	resultado
1	<code>esquerda[0]=6</code> × <code>direita[0]=3</code> → <code>6≤3?</code> não	3	0	1	[3]
2	<code>esquerda[0]=6</code> × <code>direita[1]=5</code> → <code>6≤5?</code> não	5	0	2	[3, 5]

Agora `j=2 = len(direita)` , o laço para. O que resta de `esquerda` a partir de `i=0` é **toda a lista**, `[6, 1]` — e ela é despejada no resultado **na ordem em que já estava** (sem nenhuma comparação adicional), porque `extend()` não ordena nada.

Resultado final: `[3, 5, 6, 1]` — repare que **não está ordenado** (o 1 ficou depois do 6)! Isso comprova que o algoritmo falhou silenciosamente: não houve erro, mas o resultado está errado.

- Q5.A** Primeira comparação: `6 × 3` → copia **3**
- Q5.B** Segunda comparação: `6 × 5` → copia **5**
- Q5.C** Qual lista termina primeiro? **direita**
- Q5.D** Elementos restantes: `[6, 1]` (anexados sem nenhuma comparação, na ordem original)
- Q5.E** Resultado produzido: `[3, 5, 6, 1]`

Q5.F — Conclusão compatível com a execução

B) `merge()` pressupõe que as duas listas recebidas já estejam ordenadas.

Q5.JUSTIFICATIVA

O resultado `[3, 5, 6, 1]` não está em ordem crescente, embora o código do `merge()` não contenha nenhum erro de lógica ou de sintaxe. Isso só pode ser explicado por uma **pré-condição não satisfeita**: o algoritmo assume, sem verificar, que `esquerda` e `direita` já chegam ordenadas, e compara apenas os elementos apontados por `i` e `j` a cada passo. Quando uma das listas acaba, o restante da outra é simplesmente copiado com `extend()` , sem nenhuma nova comparação — por isso, se essa lista restante estiver desordenada (como `[6, 1]`), o erro é copiado diretamente para o resultado final. Isso mostra que a correção do Merge Sort depende inteiramente de as sublistas chegarem já ordenadas ao `merge()` , o que normalmente é garantido pelas chamadas recursivas anteriores — e não pelo próprio `merge()` .

Questão 6 — Agora mude o mecanismo: Quick Sort

```
V = [10, 4, 8, 2, 9, 3, 7, 5]
```

```
def quick_sort(lista):  
    if len(lista) <= 1:  
        return lista  
    pivo = lista[-1]  
    menores, iguais, maiores = [], [], []  
    for elemento in lista:  
        if elemento < pivo:    menores.append(elemento)  
        elif elemento == pivo: iguais.append(elemento)  
        else:                 maiores.append(elemento)  
    return quick_sort(menores) + iguais + quick_sort(maiores)
```

Diferente do Merge Sort, o Quick Sort não corta a lista ao meio: ele escolhe um **pivô** (aqui, sempre o último elemento da lista) e reorganiza todos os outros elementos em três grupos, comparando cada um deles com o pivô.

Pivô = $V[-1] = 5$. Percorrendo $V = [10, 4, 8, 2, 9, 3, 7, 5]$ elemento a elemento:

Elemento	10	4	8	2	9	3	7	5
Comparado a 5	maior	menor	maior	menor	maior	menor	maior	=pivô
Vai para	maiores	menores	maiores	menores	maiores	menores	maiores	iguais

Q6

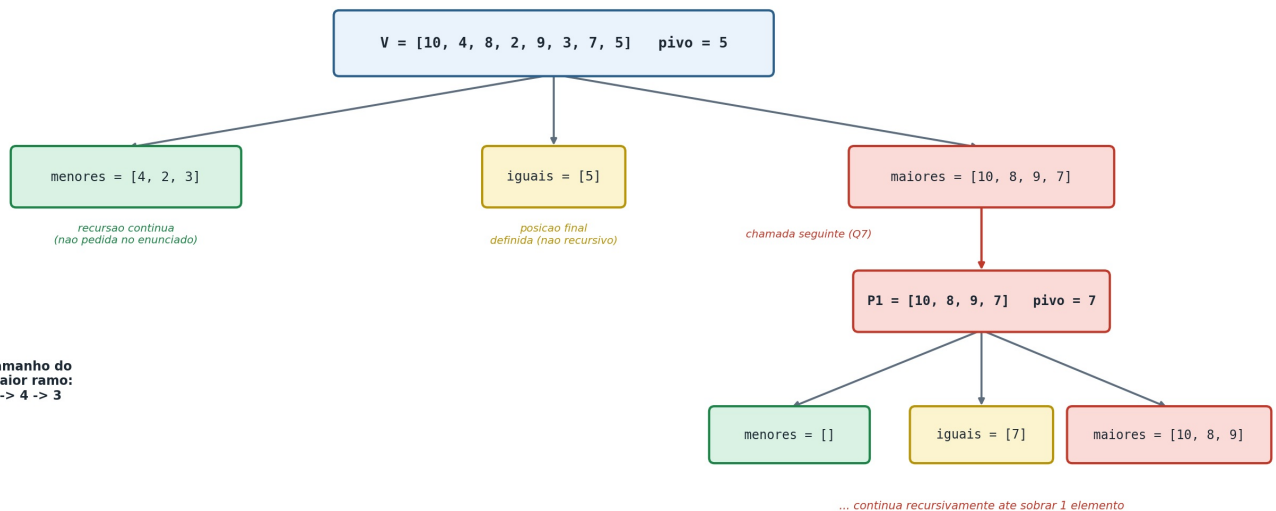


Figura 5 - Partição da Questão 6 (topo, azul) e continuação da recursão na Questão 7 (embaixo, vermelho).

Q6.A Primeiro pivô: 5

Q6.B menores = [4, 2, 3]

Q6.C iguais = [5]

Q6.D maiores = [10, 8, 9, 7]

Q6.E len(menores) = 3

Q6.F len(maiores) = 4

Q6.G — Afirmação correta

B) Os elementos são distribuídos conforme sua relação com o pivô.

(As alternativas A, C e D estão erradas: o Quick Sort não corta pelo centro, não garante partições de mesmo tamanho, e o pivô não é removido antes das comparações — ele é comparado a todos, inclusive a si mesmo.)

Questão 7 — Continue a recursão do Quick Sort

```
P1 = [10, 8, 9, 7] # maior partição obtida na Questão 6
quick_sort(P1)
```

A maior partição da Questão 6 foi `maiores = [10, 8, 9, 7]`, guardada agora como `P1`. O Quick Sort aplica a mesma lógica: pega o último elemento como novo pivô.

Pivô = `P1[-1] = 7`. Percorrendo `P1 = [10, 8, 9, 7]`:

Elemento	10	8	9	7
Comparado a 7	maior	maior	maior	=pivô
Vai para	maiores	maiores	maiores	iguais

Q7.A `P1 = [10, 8, 9, 7]`

Q7.B Novo pivô = `7`

Q7.C `menores = []`

Q7.D `iguais = [7]`

Q7.E `maiores = [10, 8, 9]`

Q7.F Trajetória do tamanho do maior ramo: `8 → 4 → 3`

Q7.G — Afirmação compatível com a execução

B) O tamanho dos próximos subproblemas depende da partição produzida pelo pivô.

Q7.JUSTIFICATIVA

Na Questão 6, a partição gerou `menores` com 3 elementos e `maiores` com 4 elementos — ou seja, o ramo `maiores` (tamanho 4) foi o maior, e é justamente ele que vira `P1` e gera a próxima chamada recursiva analisada na Questão 7. Nessa nova chamada, com pivô 7, a partição resultou em `menores=[]` (0 elementos) e `maiores=[10,8,9]` (3 elementos) — novamente o ramo `maiores` foi o maior, agora com tamanho 3. Note que o tamanho não caiu pela metade em nenhum dos dois passos (8→4 até que sim, mas 4→3 claramente não é uma divisão ao meio). Isso mostra, na prática, que a redução de tamanho no Quick Sort **depende inteiramente de onde o pivô cai dentro da lista**, e não segue uma regra fixa de "metade" como acontece no Merge Sort.

Questão 8 — IA como hipótese, não como resposta

Sugestão da IA: "Experimente usar o valor 8 como primeiro pivô para obter uma divisão mais equilibrada."

```
V = [10, 4, 8, 2, 9, 3, 7, 5]
pivo_teste = 8
```

Vamos testar a hipótese na prática, usando a mesma função `particionar`. Percorrendo `V` e comparando cada elemento a 8:

Elemento	10	4	8	2	9	3	7	5
Comparado a 8	maior	menor	=pivô	menor	maior	menor	menor	menor
Vai para	maiores	menores	iguais	menores	maiores	menores	menores	menores

Q8.A menores = `[4, 2, 3, 7, 5]`

Q8.B iguais = `[8]`

Q8.C maiores = `[10, 9]`

Q8.D Diferença absoluta entre `len(menores)` e `len(maiores)` usando pivô 8: $|5 - 2| = 3$

Q8.E Na questão 6, com pivô 5, essa diferença foi: $|3 - 4| = 1$

Comparacao de equilibrio da particao em V = [10,4,8,2,9,3,7,5]

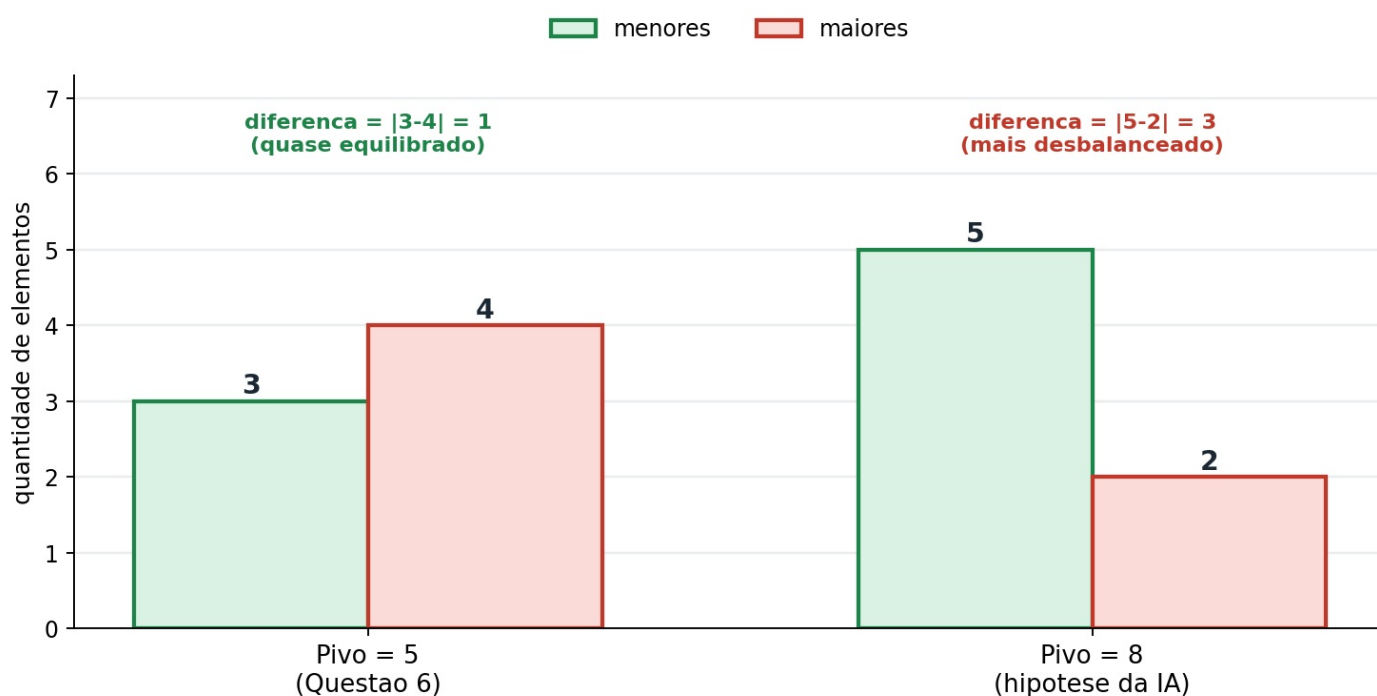


Figura 6 – Quanto menor a diferença entre os dois lados, mais equilibrada é a partição. Pivô 5 (diferença 1) supera o pivô 8 sugerido pela IA (diferença 3).

Q8.F — Conclusão sustentada pelos dados

A) O pivô 5 produziu a primeira partição mais equilibrada neste vetor.

Q8.JUSTIFICATIVA

A sugestão da IA **não foi confirmada**. Os dados mostram exatamente o oposto do que foi previsto: usar 8 como pivô gerou uma partição de tamanhos 5 e 2 (diferença de 3), enquanto o pivô 5, já usado na Questão 6, gerou uma partição de tamanhos 3 e 4 (diferença de apenas 1) — muito mais próxima do ideal (partições de tamanho igual). Isso reforça a orientação da própria prova: **uma IA pode sugerir um caminho, mas cabe a quem programa testar e verificar** antes de aceitar a sugestão como verdadeira. Aqui, o teste prático contradisse a hipótese.

Questão 9 — Teste a conclusão em outra entrada

```
W = [3, 6, 9, 12, 15, 18, 21, 24] # já ordenado!
```

```
pivo_A = W[-1] # = 24  
pivo_B = 15
```

Repare que **W** já está **ordenado**. Isso é o ingrediente-chave desta questão: quando a lista já está ordenada e o pivô escolhido é sempre o último elemento, esse pivô será **sempre o maior valor** da lista — o pior cenário possível para o Quick Sort.

Estratégia A — pivô = último elemento (24)

Como 24 é o maior valor de **W**, todos os outros 7 elementos são menores que ele.

Q9.A Primeiro pivô: 24

Q9.B menores = [3, 6, 9, 12, 15, 18, 21]

Q9.C maiores = []

Q9.D — Se essa escolha se repetir em listas já ordenadas...

Se aplicarmos o Quick Sort de novo em **menores** = [3,6,9,12,15,18,21] (também ordenada), o último elemento (21) volta a ser o maior, produzindo **menores** com 6 itens e **maiores** vazio — e assim por diante, perdendo **apenas 1 elemento por chamada**.

B) uma cadeia $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots$

Estratégia B — pivô = 15 (valor do meio)

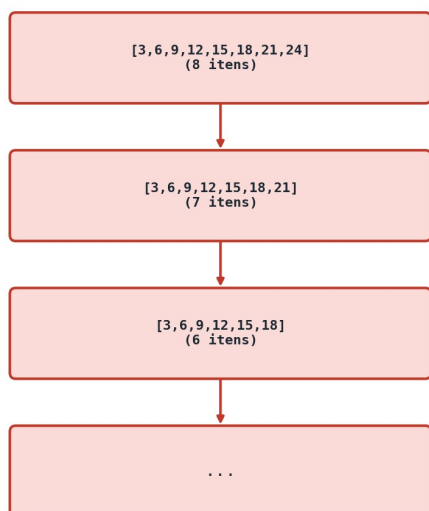
Agora escolhendo 15 (que está mais perto do centro do vetor) como pivô:

Q9.E menores = [3, 6, 9, 12]

Q9.F iguais = [15]

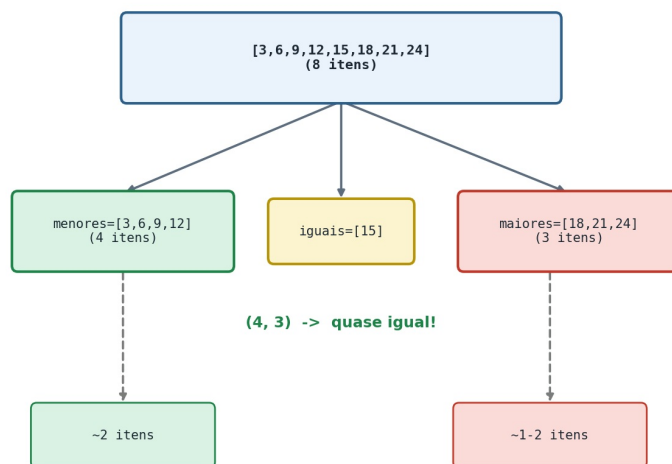
Q9.G maiores = [18, 21, 24]

ESTRATEGIA A - pivô = ultimo elemento (24)



Cadeia $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots$
profundidade $O(n)$ = PIOR CASO

ESTRATEGIA B - pivô = 15 (valor do meio)



(ilustrativo) se a proporção se mantivesse, a árvore ficaria bem mais BAIXA e BALANCEADA

Profundidade aproximada $O(\log n)$ = CASO (quase) IDEAL

Figura 7 – Comparação direta: pivô no extremo de uma lista ordenada gera uma cadeia longa (pior caso); pivô próximo do centro gera uma árvore muito mais baixa e equilibrada.

Q9.H — Qual estratégia produz a partição mais equilibrada em W?

Estratégia B

Q9.JUSTIFICATIVA

A Estratégia A (pivô = 24, o último elemento) produziu uma partição de **(7, 0)** — o pior desequilíbrio possível, já que todo o vetor caiu de um único lado. A Estratégia B (pivô = 15) produziu uma partição de **(4, 3)**, quase perfeitamente equilibrada. Isso acontece porque **W** já está ordenado: usar sempre o último elemento como pivô, nesse cenário, sempre seleciona o maior valor restante, então cada chamada recursiva só remove **um elemento** (o pivô) da lista, formando uma cadeia longa e estreita — exatamente o formato descrito na alternativa B da Questão 9.D ($n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots$), que corresponde ao **pior caso** do Quick Sort, com profundidade proporcional a n em vez de $\log n$. Já a Estratégia B, ao escolher um valor mais próximo da mediana dos dados, divide o vetor em dois ramos de tamanho parecido a cada chamada, produzindo uma árvore de recursão muito mais baixa e equilibrada — o comportamento desejado, mais próximo do caso médio/ideal do algoritmo.

Resumo geral para revisão rápida

Questão	Tema central	Ideia principal para lembrar
1	Recursão simples vs. dupla	Uma chamada recursiva por nível = cadeia linear; duas chamadas por nível = árvore binária.
2	Divisão do Merge Sort	A lista é sempre cortada pela posição (<code>len//2</code>), nunca pelo valor dos elementos.
3	Descida x retorno	Descida = divisão (sem comparações); retorno = merge (é aqui que a ordenação de fato acontece).
4	merge() correto	Dois ponteiros avançam comparando o topo de cada lista; o que sobra é só copiado no final.
5	Pré-condição do merge()	merge() só funciona corretamente se ambas as entradas já estiverem ordenadas — ele não verifica isso.
6 e 7	Partição do Quick Sort	O tamanho dos subproblemas depende de onde o pivô cai; não há garantia de divisão pela metade.
8	Validar hipóteses de IA	Sempre testar sugestões de IA com dados reais antes de aceitar a conclusão.
9	Melhor x pior caso do Quick Sort	Pivô no extremo de uma lista já ordenada = pior caso (cadeia); pivô perto da mediana = melhor caso (árvore balanceada).

Guia gerado a partir da prova "DSA - Avaliação: Recursividade, Merge Sort e Quick Sort" (Turma 4) e da conversa de resolução em chat.
Material de apoio ao estudo — não substitui a leitura do enunciado original nem a prática de execução do código.